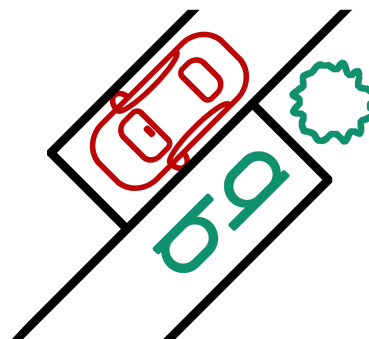
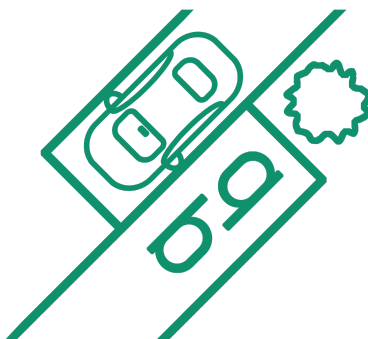
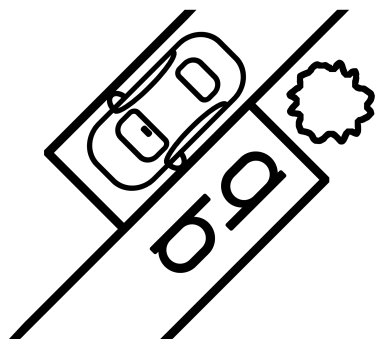


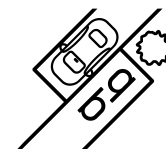
Proyecto intermodular - Memoria completa

Aplicación de gestión para la localización de plazas libres de parking



Mario Fernández y Edwin Varón

10 de mayo del 2026



Índice

Índice	1
Parte I. Descripción General	4
Resumen general	4
Abstract	4
Descripción	4
Descripción del proyecto	4
Objetivo general	4
Justificación	5
Logo del proyecto	5
Desarrollo	5
Resultados	6
Esquema general de funcionamiento	6
Ventanas	6
Arquitectura	17
Funcionalidades principales / Alcance	18
Análisis de la competencia	18
Parkopedia	18
EasyPark	19
Análisis DAFO	20
Debilidades	20
Amenazas	21
Fortalezas	21
Oportunidades	21
Marco tecnológico	22
Software	22
Hardware	23
Parte II. Análisis y Diseño técnico	23
Requisitos	23
Requisitos funcionales	23
Requisitos no funcionales	24
Diseño físico/lógico de datos	25
Diagrama Entidad-Relación (E-R)	26
Diseño funcional	27
Casos de uso	27
Diagrama de clases	30

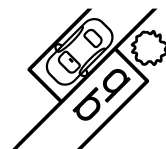
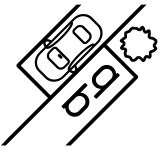
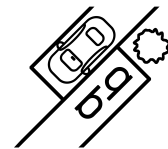


Diagrama de secuencia	31
Diseño de interfaces	34
Visualizar mapa	34
Menú desplegable	34
Pantalla de acceso al menú de administración	34
Menú de administración	34
Alta de parking	34
Visualización de estadísticas	35
Material necesario para llevar a cabo el proyecto	35
Personal	35
Equipo	35
Manuales	35
Parte III. Planificación y memoria económica	36
Gantt	36
Recursos	36
Costes	38
Áreas de gestión del proyecto	39
Gestión de riesgos	39
Gestión del equipo del proyecto (personal)	39
Jefe de proyecto	39
Analista	40
Desarrollador de BBDD	40
Desarrollador backend	40
Desarrollador frontend	40
Técnico de sistemas	40
Organigrama	41
Parte III. Implementación y pruebas	42
Grado de consecución producto	42
Comparativa Gantt inicio y final	42
Grado de cumplimiento de requisitos funcionales	43
Existencia de un prototipo funcional	43
Pruebas realizadas	44
Mapa de navegación: Capturas de la aplicación y conexión entre ellas	45
Innovación	46
Innovación	46
Funcionalidades potenciales	46
Resultados y conclusiones	47
Bibliografía - Webgrafía	48



Competencia	48
Arquitectura	48
Análisis técnico	48
Desarrollo backend	48
Desarrollo frontend	49
Anexos	50
Anexo 1 - Código SQL	50



Parte I. Descripción General

Resumen general

Abstract

This project proposes the design and development of a web application, named "QuicklyPark", for visualizing the parking spaces in real time. Through a configuration file, the system models the parking lot and allows clients to visualize a map showing the availability of spaces. Also, it provides managers with an administration panel to visualize daily usage statistics and modify or add new parking lots. The system implements a three-tier architecture integrating a frontend in Vue.js, a backend developed in Java with Spring Boot, and a MySQL database, communicating to update the status of the spaces in real time.

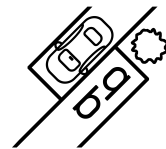
Descripción

Este proyecto propone el diseño e implementación de una aplicación web, llamada "QuicklyPark", para la visualización de plazas de parking en tiempo real. Mediante un fichero de configuración, el sistema modela el aparcamiento y permite a los clientes visualizar un mapa con la disponibilidad de las plazas. Además, proporciona a los gestores un panel de administración para visualizar estadísticas de uso diarias y modificar o añadir nuevos parkings. El sistema utiliza una arquitectura de tres capas que integra un frontend en Vue.js, un backend desarrollado en Java con Spring Boot, y una base de datos MySQL, comunicándose para actualizar el estado de las plazas en tiempo real.

Descripción del proyecto

Objetivo general

Creación de un simulador web sobre la disponibilidad de plazas libres en un parking. Mediante un fichero de configuración se modelará el parking en cuestión y aparecerá dibujado en una aplicación web, mostrando la vista de planta. Esta aplicación estará disponible para los clientes, facilitando su aparcamiento, mostrando aquellas plazas que



estén libres en cada momento. Las plazas se irán ocupando y desocupando en tiempo real. A su vez, proporcionará estadísticas de uso diarias.

La idea es que los clientes puedan visualizar el mapa en una aplicación web en su móvil o en pantallas distribuidas por el parking. El propio sistema de plazas del parking, deberá comunicar vía API, cuando se ocupa o libera una plaza, con esta aplicación.

Se ha decidido utilizar como nombre “**Quickly Park**”, haciendo referencia a la optimización del tiempo a la hora de encontrar plaza libre en un parking.

El **objetivo general** es optimizar el tiempo de aparcamiento, indicando la disponibilidad de las **plazas** dentro de un **mapa** en **tiempo real**.

Justificación

La búsqueda de **plazas libres** en parkings puede volverse **estresante**; en días de alta actividad es complicado encontrar las plazas libres, y no solo es **problemático** para los **conductores que buscan aparcamiento**. Cuando hay escasez de plazas, los coches se desesperan buscando sin rumbo fijo y generan **atascos internos**, por lo que los coches que quieren salir también sufren de este problema.

De cara a los **gestores** del aparcamiento, recoger datos y ver estadísticas es muy **importante** porque les ayuda a entender **qué pasa realmente** en el parking. Así pueden ver cuándo se llena más, cuándo hay menos coches y qué **problemas se repiten**, y con esa información pueden **organizar mejor las plazas**, evitar líos de tráfico y hacer que todo funcione de forma más sencilla.

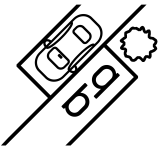
Logo del proyecto

Se han desarrollado una serie de logos e iconos en diferentes colores, para poder ser implementados en diversos entornos.

Desarrollo

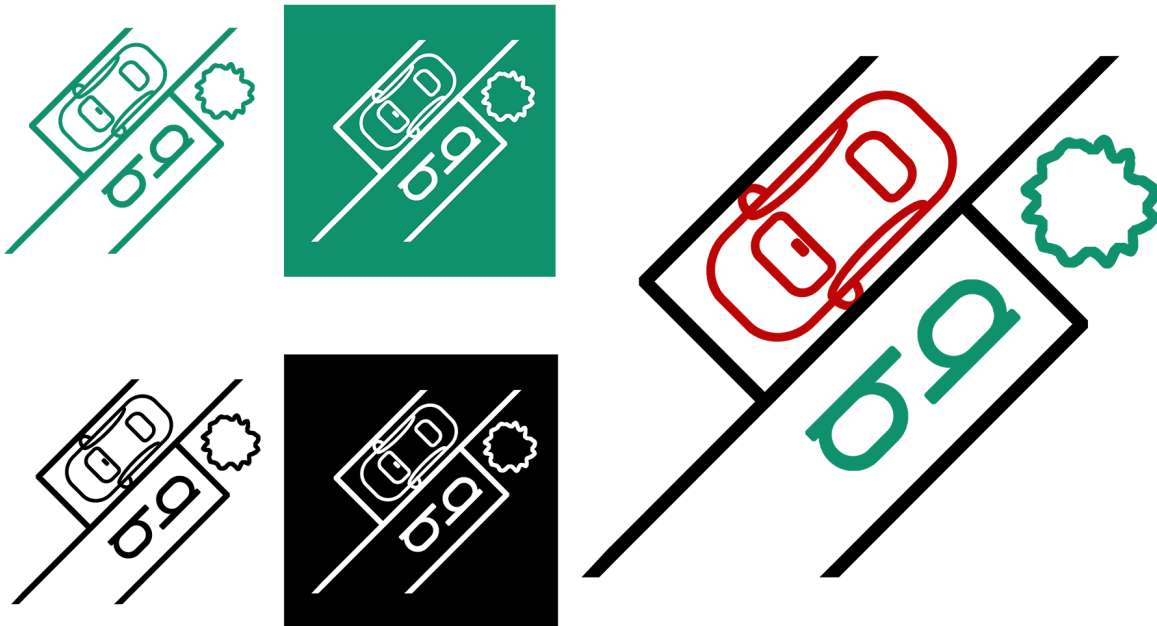
Se inspira en el nombre de la aplicación: “**Quickly Park**”.

En el logotipo podemos encontrar dos plazas de parking formando un rayo, que representa la facilidad y rapidez de encontrar sitio gracias a nuestro servicio. Los colores utilizados



hacen referencia al estado de ocupación de las plazas del parking. Además podemos encontrar las iniciales del proyecto (q y p).

Resultados



Esquema general de funcionamiento

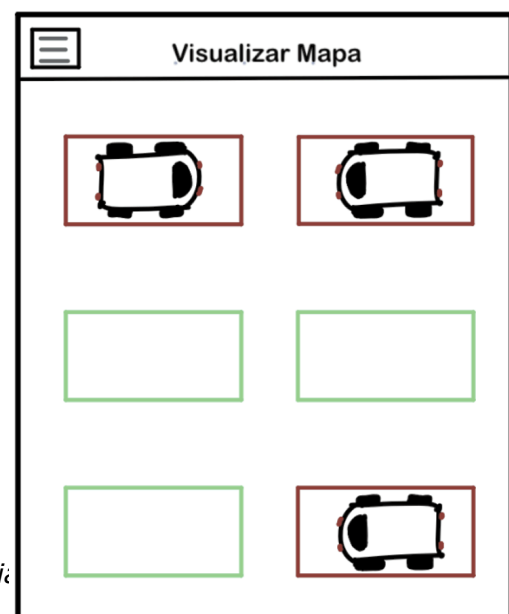
Ventanas

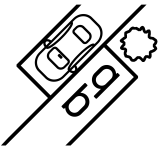
Visualizar mapa

Es la ventana que el usuario encuentra al abrir la aplicación, además de la más importante. Muestra el mapa de ocupación del parking coloreado según el estado de cada plaza.

Pueden acceder a ella:

- Clientes del parking
- Gestores del parking







Visualizar mapa

Seleccione un parking:

Seleccionar ▼

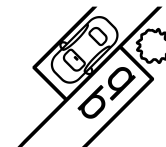
Horario: 24h

Dirección: Calle Mayor 10, Zaragoza

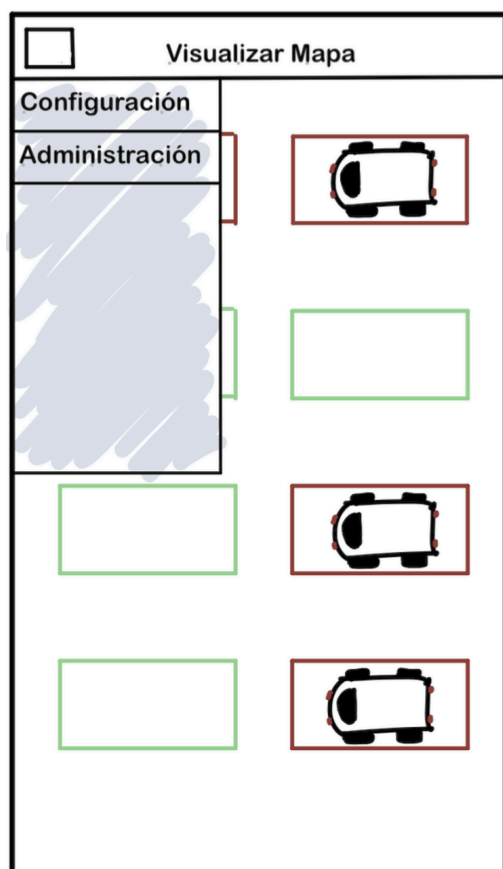
	
	
	

Permite:

- Visualizar las **plazas de parking**, en **rojo** y con un icono de un coche si están ocupadas; y en **verde** si están libres.
- Desplegar el **menú de navegación** de la aplicación.



Menú desplegable



Es el menú que se despliega desde la parte superior izquierda de la pantalla.

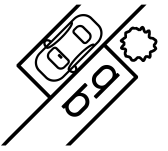
Pueden acceder a ella:

- Clientes del parking
- Gestores del parking

Permite:

• Navegar por las distintas pantallas de la aplicación.

- Configuración
- Administración





Visualizar mapa

 Inicio

 Configuración

 Administración

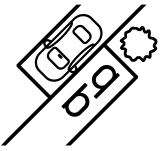
Seleccione un parking:

Seleccionar ▼

Horario: 24h

Dirección: Calle Mayor 10, Zaragoza



Pantalla de acceso al menú de administración

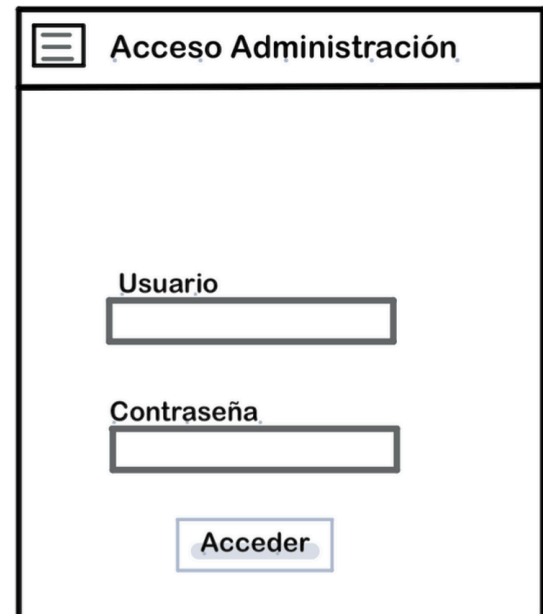
Contiene el formulario que permite iniciar sesión a los gestores o administradores de la aplicación.

Pueden acceder a ella:

- Gestores del parking

Permite:

- Iniciar sesión a gestores
- Acceder al menú de administración

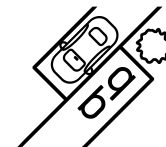


Acceso Administración

Usuario

Contraseña

Acceder





Acceso administración

 Inicio

 Configuración

 Administración

Usuario

Contraseña

 **Menú Administración**

Alta Parking

Estadísticas

Menú de administración

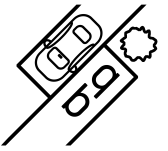
Permite navegar entre las diferentes pantallas del menú de administración.

Pueden acceder a ella:

- Gestores del parking

Permite:

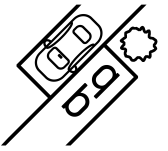
- Acceder a las estadísticas
- Acceder a la gestión de parkings



Menú Administración

Alta Parking

Estadísticas



Alta de parking

Es la pantalla destinada a dar de alta nuevos parkings a través de un fichero de texto.

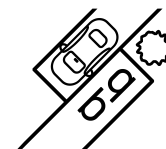
Pueden acceder a ella:

- Gestores del parking

Permite:

- Cargar ficheros para crear nuevos parkings



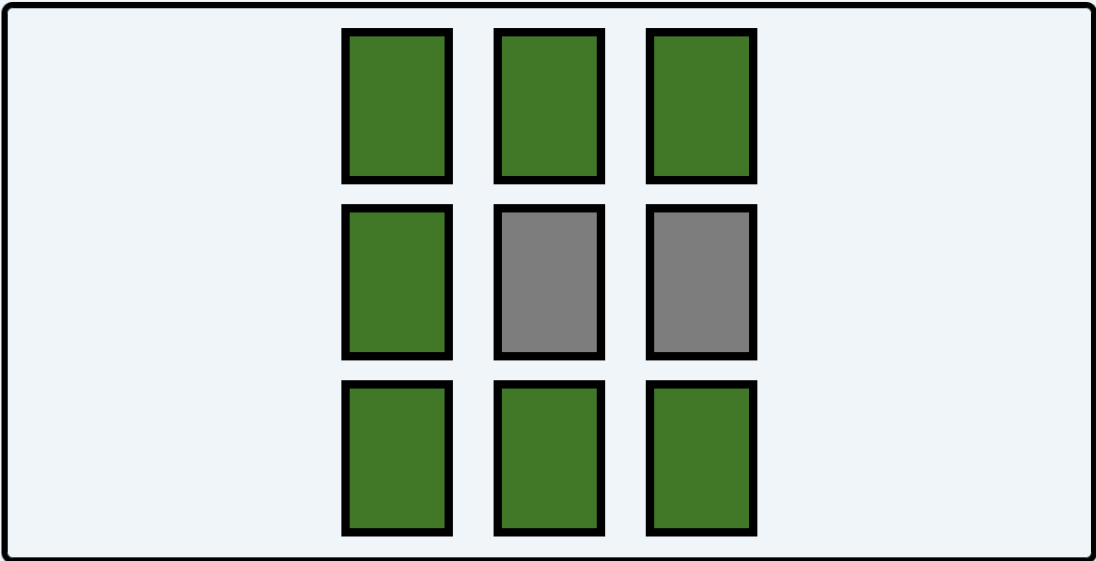




Alta parking

parking1.txt

Previsualización



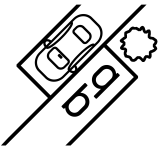
Dirección:

Calle de prueba

Horario:

24H

Añadir parking



Visualización de estadísticas

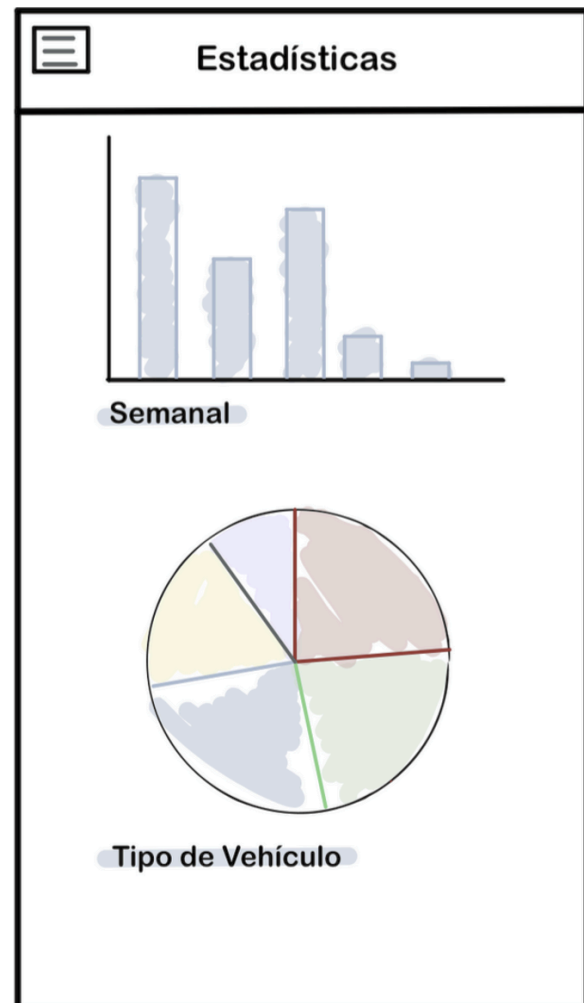
Muestra las **estadísticas** de uso del parking a través de gráficos y tablas.

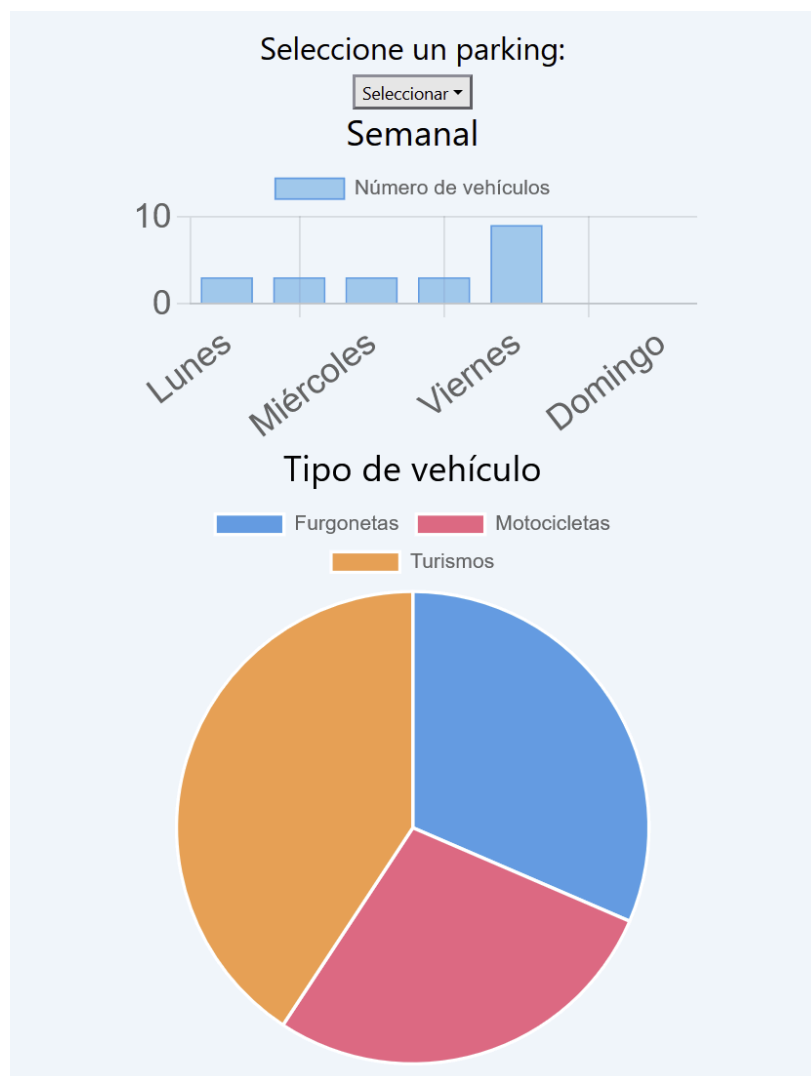
Pueden acceder a ella:

- Gestores del parking

Permite:

- Visualizar estadísticas de uso del parking





Configuración

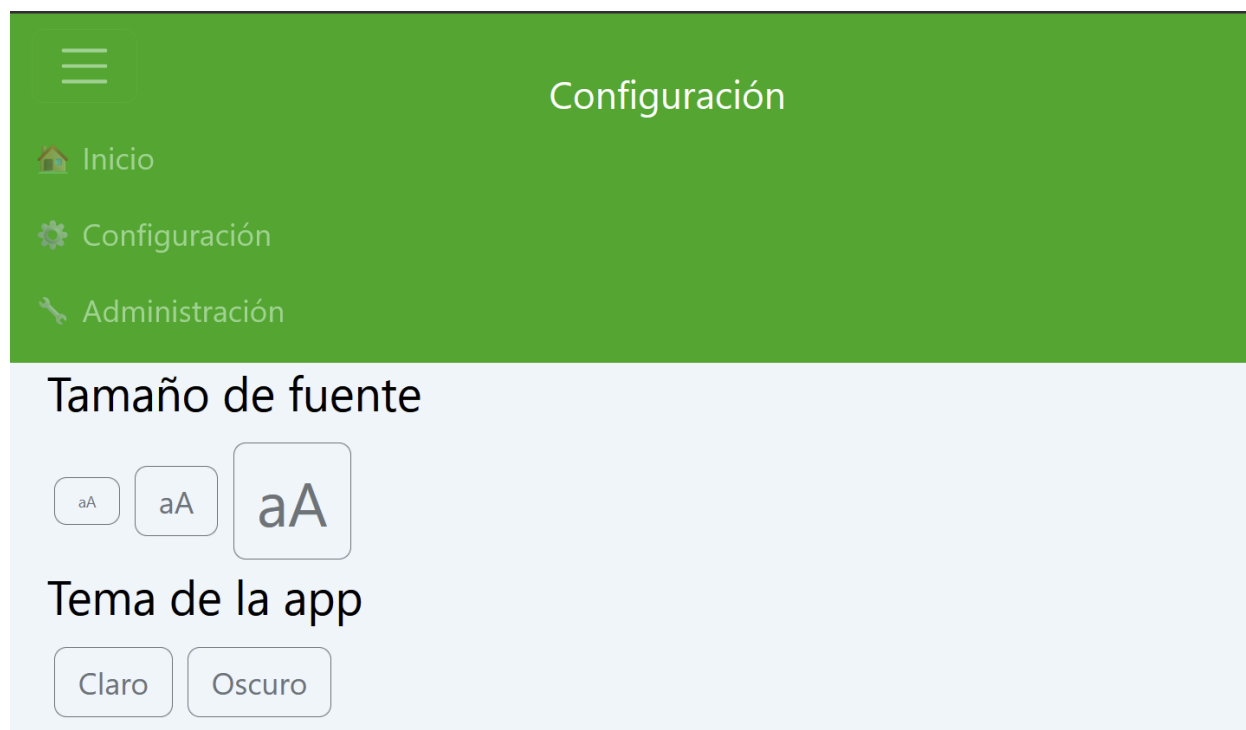
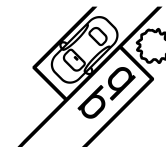
Permite cambiar ajustes de la aplicación.

Pueden acceder a ella:

- Clientes del parking
- Gestores del parking

Permite:

- Configurar el tamaño de fuente y el tema de la aplicación

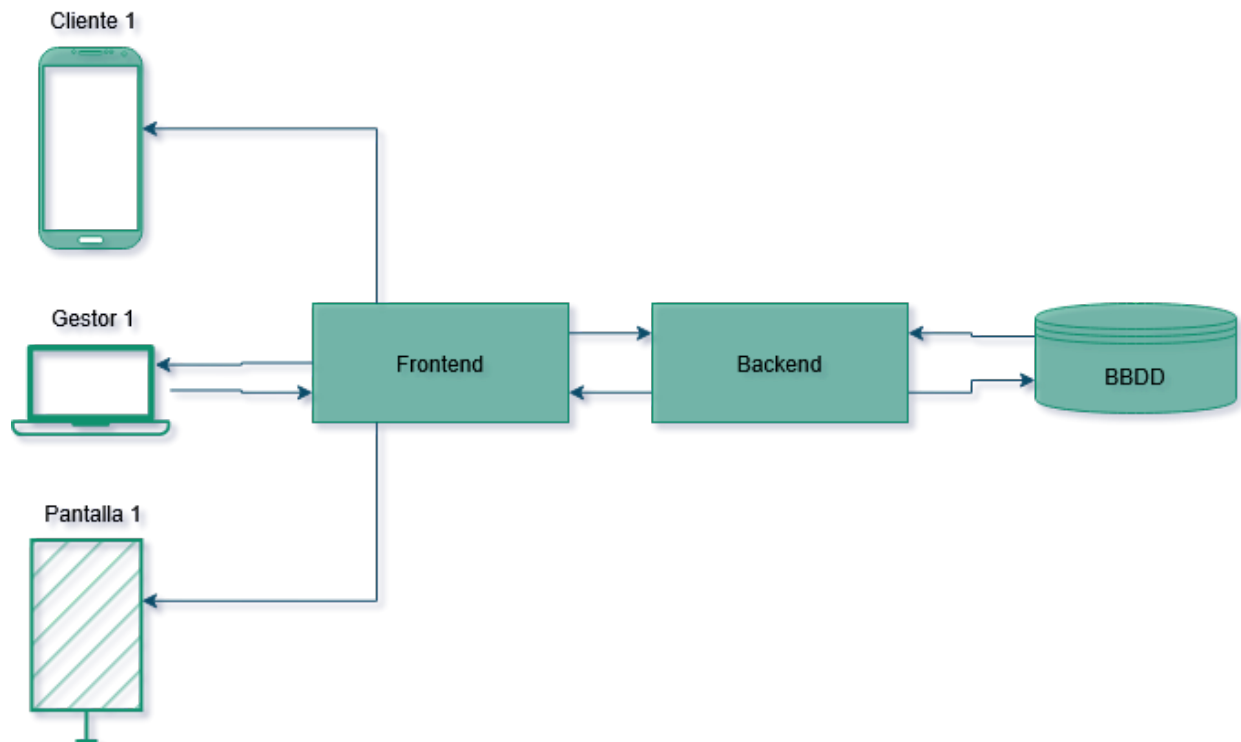
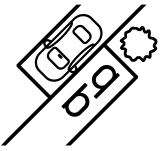


Arquitectura

Se ha decidido utilizar una arquitectura de tres niveles:

- Nivel de datos (la base de datos)
- Nivel de aplicación (el backend)
- Nivel de presentación (el frontend)

El sistema utiliza una arquitectura de tres capas donde diversos dispositivos (móviles, pantallas y ordenadores de gestión) se conectan a un frontend. Este frontend actúa como intermediario, recogiendo las peticiones de los usuarios para enviarlas al backend. El backend es el componente principal, procesando toda la lógica de negocio del parking. Finalmente, este se comunica de forma constante con la base de datos (BBDD) para almacenar, consultar y actualizar la información en tiempo real.



Funcionalidades principales / Alcance

- **Área clientes:**
 - No necesita identificación.
 - Permite visualizar el mapa de ocupación en tiempo real del parking.
- **Panel de administración:**
 - Necesita identificación.
 - Permite modificar y añadir parkings a través de un fichero de texto.
 - Permite visualizar estadísticas del uso del parking.

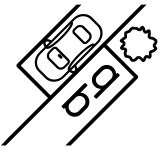
Análisis de la competencia

Se han analizado dos servicios de la competencia: "EasyPark" y "Parkopedia".

Parkopedia

- **Descripción:** Base de datos global de parkings con disponibilidad en tiempo real.





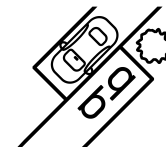
- **Fortalezas:** Amplia cobertura, integración con fabricantes de autos y apps de navegación.
- **Debilidades:** No siempre ofrece datos precisos en tiempo real; interfaz centrada en búsqueda, no en análisis visual.

EasyPark



- **Descripción:** App móvil para pago y localización de plazas libres.
- **Fortalezas:** Alta adopción en ciudades europeas, integración con sistemas municipales.
- **Debilidades:** no ofrece visualización analítica o mapas de ocupación.

En resumen, **ninguna** permite visualizar mapas de **ocupación en tiempo real**, que es lo que se centrará el proyecto.

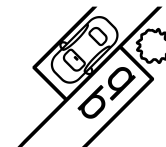


Análisis DAFO



Debilidades

1. **Coste y complejidad:** requiere una infraestructura más avanzada y mantenimiento continuo para garantizar su fiabilidad.
2. **Errores en la visualización:** posibles errores en la simulación si la configuración del parking es incorrecta.
3. **Fallos de sincronización:** posibles fallos de sincronización en tiempo real si hay alta concurrencia.
4. **Formación:** requiere personal con conocimientos técnicos para su gestión.



Amenazas

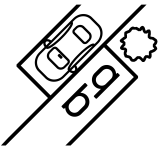
1. **Competencia consolidada:** existen plataformas con muchos usuarios que podrían ofrecer funciones similares.
2. **Obsolescencia tecnológica:** los avances rápidos pueden dejar anticuado el sistema si no se actualiza con frecuencia.
3. **Cobertura:** es posible que algunos parkings no tengan cobertura dentro.
4. **Dependencia tecnológica:** si los sensores o el sistema de comunicación fallan, los datos mostrados pueden ser incorrectos.
5. **Retraso en el desarrollo:** si se ha estimado mal, puede retrasarse la entrega al cliente.
6. **Abandono de un miembro del equipo:** una baja o imprevisto puede perjudicar al desarrollo.

Fortalezas

1. **Visualización en tiempo real:** permite a los usuarios saber exactamente qué plazas están libres, reduciendo el tiempo de búsqueda y mejorando la experiencia.
2. **Fácil acceso:** el sistema puede verse desde móviles o pantallas del parking, sin necesidad de iniciar sesión.
3. **Adaptable:** el sistema se configura fácilmente mediante un fichero de texto.
4. **Implementación:** el sistema se implementa de forma sencilla, los parkings ya cuentan con un sistema de plazas y solo hay que conectarse a esos datos.

Oportunidades

1. **Crecimiento de las smart cities:** los ayuntamientos y empresas buscan soluciones inteligentes para gestionar el tráfico y el aparcamiento.
2. **Integración futura:** puede ampliarse con funciones como pago automático o reserva de plazas, aumentando su utilidad.
3. **Interés:** de centros comerciales, hospitales y parkings privados en optimizar espacios.
4. **Escalabilidad:** es sencillo escalar el sistema a múltiples parkings desde una misma plataforma.



Marco tecnológico

Software

Frontend

- **HTML5, CSS3 y JavaScript:** tecnologías base para el desarrollo de la interfaz web.
- **Framework sugerido:** *Vue.js*, por su capacidad de actualizar la interfaz en tiempo real y mantener un código modular y eficiente.
- **Comunicación con el backend:** a través de **API REST** o **WebSockets** para la actualización instantánea de la disponibilidad de plazas.
- **Compatibilidad:** adaptable a dispositivos móviles (responsive design) y pantallas grandes de información.

Backend

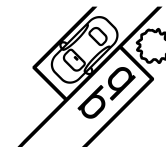
- **Lenguaje y framework:** **Java** con **Spring Boot**, adecuado por su rendimiento y facilidad para crear servicios RESTful. También se ha utilizado **Swagger** para testear la API.
- **Autenticación:** sistema de acceso seguro para el **panel de administración** mediante **JWT** (JSON Web Tokens).
- **IDE:** Eclipse 2026-03.

Base de datos (BBDD)

- **Sistema propuesto:** *MySQL*, bases de datos relacionales robustas y seguras.
- **Datos almacenados:** configuración del parking, estado actual de las plazas, histórico de ocupación y estadísticas diarias.

Herramientas adicionales

- **Docker:** para la contenedorización y despliegue del sistema en distintos entornos.
- **GitHub:** control de versiones y colaboración en equipo.
- **Photoshop:** para el desarrollo del logotipo e iconos.
- **Postman:** para testear la API.



Hardware

Servidor

- Procesador **multinúcleo** (mínimo 4 núcleos) y **8 GB de RAM** para gestionar peticiones concurrentes.
- Almacenamiento **SSD** para mejorar la velocidad de acceso a la base de datos.
- Conectividad **estable a Internet** para ofrecer datos en tiempo real.

Dispositivos cliente

- **Cliente y gestor:** cualquier dispositivo con **navegador web** moderno (ordenador portátil, tablet o smartphone).
- **Pantallas informativas:** monitores o paneles táctiles conectados a un **mini PC** o **Raspberry Pi** que muestre la interfaz web en modo pantalla completa.

Parte II. Análisis y Diseño técnico

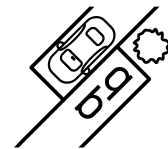
Requisitos

Los requisitos funcionales son las funciones y tareas específicas que el sistema debe realizar para cumplir su objetivo.

Los requisitos no funcionales son las características de calidad que determinan cómo debe funcionar el sistema, como velocidad, seguridad o facilidad de uso.

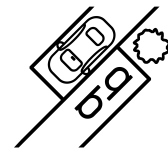
Requisitos funcionales

- Los gestores podrán modelar el parking a partir de un **fichero de texto**.
- El gestor podrá visualizar **estadísticas de uso** diarias recogidas por el sistema a través de gráficos.
- Los clientes podrán visualizar un **mapa de ocupación** del parking en **tiempo real**.
- La aplicación guardará los registros en una **base de datos**.



Requisitos no funcionales

- El sistema debe permitir almacenar hasta **1000 parkings de hasta 5000 plazas**.
- El mapa debe actualizarse en **tiempo real**.
- La aplicación debe ser **accesible para personas con limitaciones físicas o sensoriales**: en configuración debe existir un apartado de **accesibilidad**. Pudiendo, aumentar contraste, ajustar la gama de colores para personas con daltonismo, adaptar el tamaño de la letra y navegar por la aplicación mediante teclado.
- La aplicación debe separar sus capas en **módulos** independientes.
- El código se gestionará mediante un **control de versiones** (Git) .
- La aplicación debe poder visualizarse bien en **ordenadores y móviles**.
- La aplicación debe realizar **backups** varias veces al día.
- La aplicación debe volver a **desplegarse automáticamente** en caso de caída.
- La aplicación debe ser **intuitiva** y fácil de usar.



Diseño físico/lógico de datos

Para el diseño de los datos se han creado cuatro entidades:

- **Parking**: representa a cada uno de los parkings que puede gestionar la aplicación.
- **Plaza**: representa cada una de las casillas de un parking.
- **Gestor**: son los usuarios gestores de la aplicación.
- **Registro**: guarda información de uso de las plazas.
- **Tipo vehículo**: almacena los tipos de vehículos registrados en la aplicación.

Estas, se relacionan de la siguiente forma:

- Un **Parking** puede contener una o varias **Plazas**, y a su vez una **Plaza** solo puede pertenecer a un único **Parking** en específico, lo que daría una relación **1:N**.
- Una **Plaza** puede tener asociados **uno o varios Registros** a lo largo del tiempo (diferentes vehículos aparcando en distintos momentos), pero un Registro de estacionamiento concreto se asocia a **una única Plaza**, por lo que sería una relación **1:N**.
- Un **Tipo_vehículo** (por ejemplo, coche, moto) puede estar asociado a **uno o varios Registros**, y a su vez un **Registro** pertenece a un **único Tipo_vehículo**, lo que conforma otra relación **1:N**.
- La entidad **Gestor** actúa de forma **independiente** en cuanto a relaciones directas en este esquema, ya que representa a los administradores del sistema que gestionan la aplicación y tienen permisos de acceso, pero no interactúan directamente a nivel de base de datos.

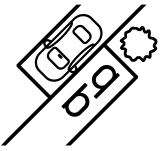
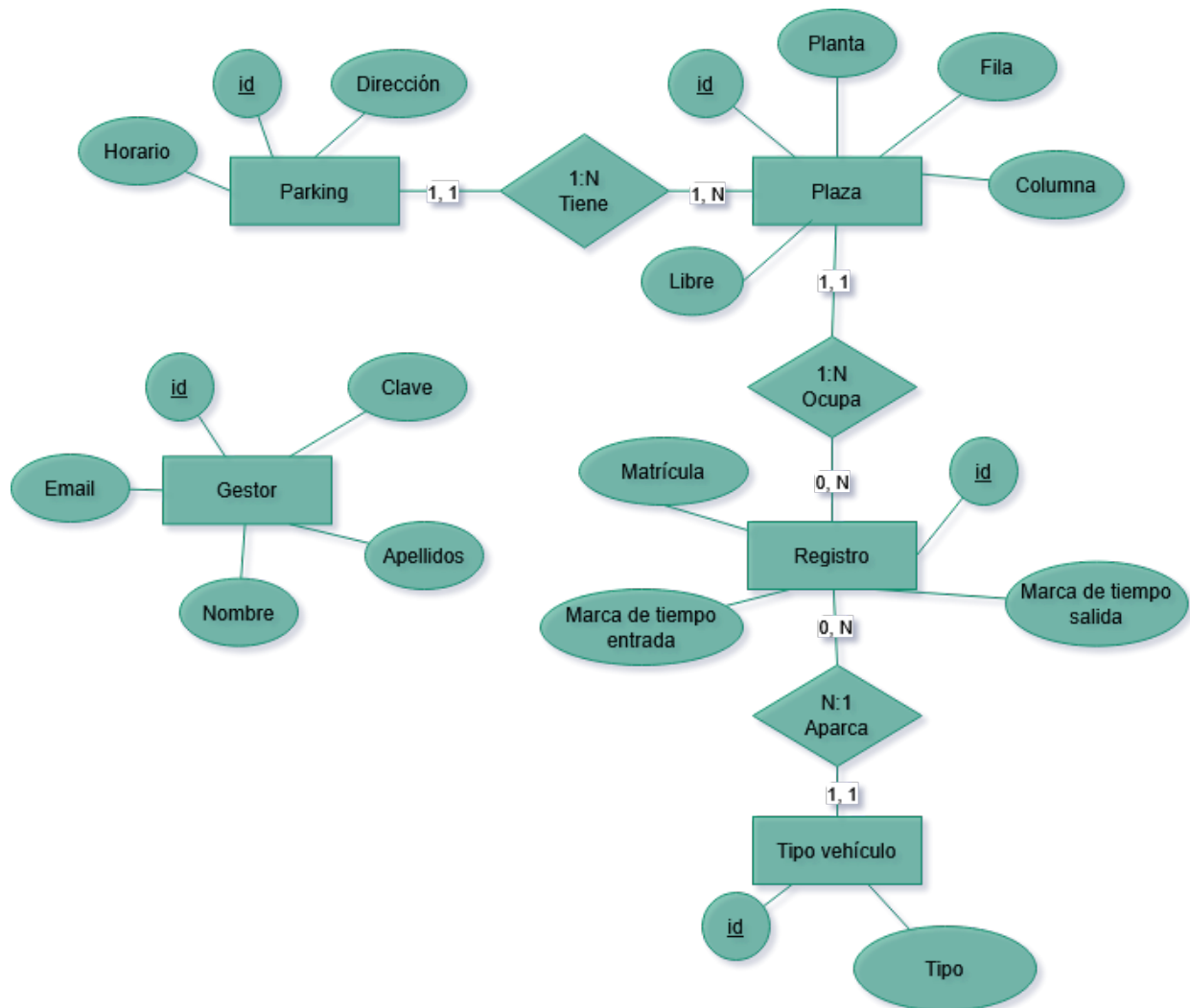
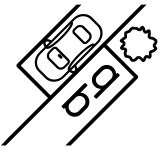


Diagrama Entidad-Relación (E-R)



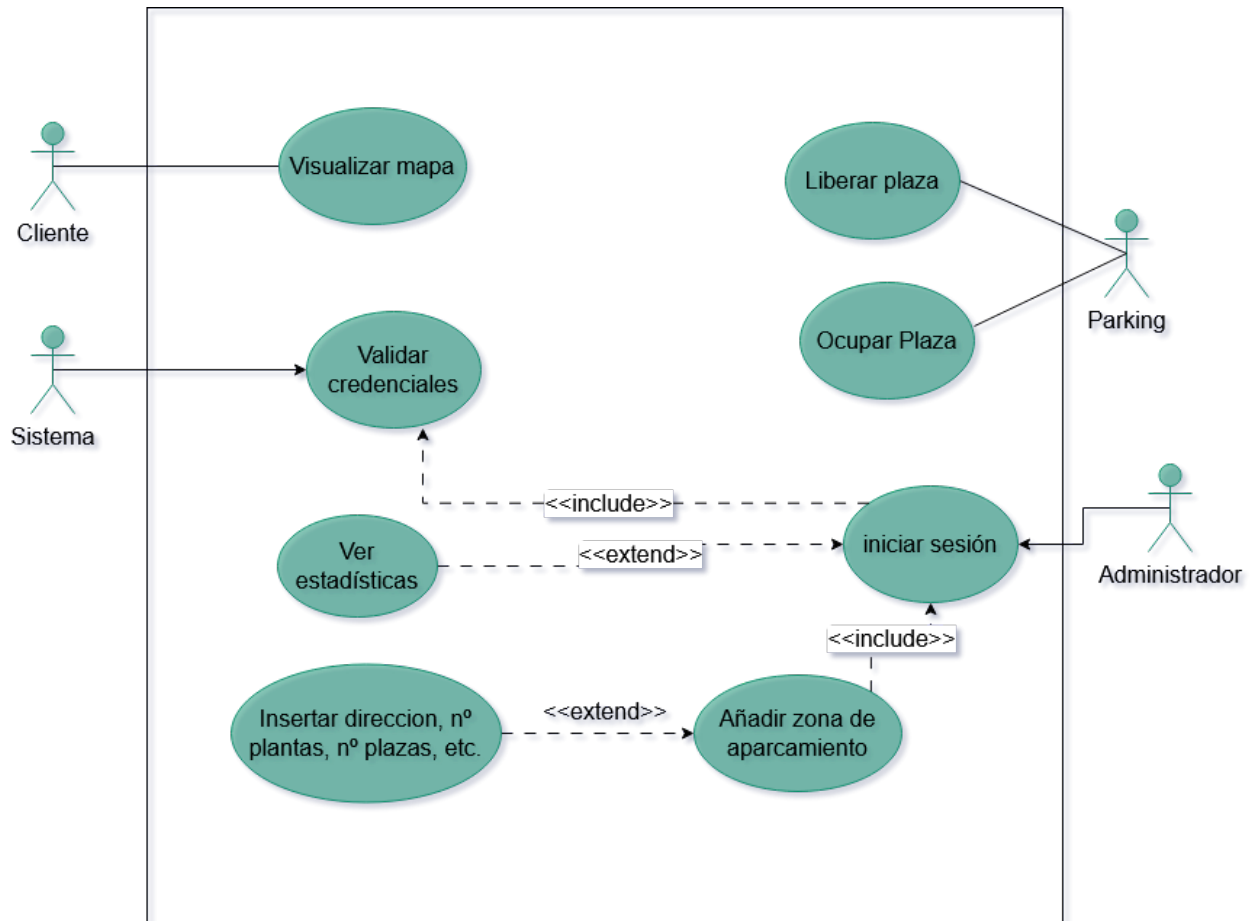
[Código SQL](#). (También disponible en **Anexo 1 - Código SQL**)



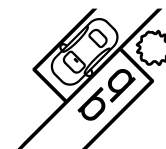
Diseño funcional

Casos de uso

Este diagrama representa los diferentes casos de uso de la aplicación, más abajo se describe cada uno de ellos.

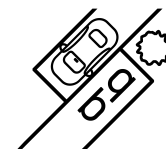


Caso de uso: Visualizar disponibilidad del parking	
ID: CU-1	
Descripción	El usuario desea consultar en tiempo real las plazas disponibles en un parking a través de la aplicación web.
Actores	Cliente (usuario del parking)

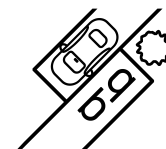


Precondiciones	- El sistema debe tener configurado el parking y conectado a la base de datos. - Deben existir datos actualizados sobre la ocupación.
Flujo normal	- El usuario accede a la aplicación web desde su dispositivo móvil o una pantalla informativa. - El frontend solicita al backend la información actual del estado de las plazas. - El backend obtiene los datos de la base de datos. - El frontend muestra el mapa del parking, indicando las plazas libres y ocupadas.
Postcondiciones	El usuario visualiza correctamente la ocupación actual del parking.
Flujo alternativo	- No hay conexión con el servidor o base de datos → se muestra un mensaje de error. - No existen datos del parking → se muestra un mensaje de error.

Caso de uso: Administrar parking	
ID: CU-2	
Descripción	El gestor desea añadir, eliminar o modificar parkings a través de un fichero de configuración.
Actores	Administrador
Precondiciones	- El gestor debe estar autenticado en el panel de administración. - Debe existir conexión con la base de datos.
Flujo normal	- El gestor accede al panel de administración. - Selecciona la opción "Gestionar parkings". - Sube o edita un fichero de configuración con la estructura del parking.



	4. El backend procesa el fichero y actualiza los datos en la base de datos. 5. El sistema confirma la actualización.
Postcondiciones	La base de datos contiene la información actualizada del parking.
Flujo alternativo 1 - Fallo de validación en el formato del fichero	• Condición de activación: El sistema detecta que el fichero subido no cumple con la codificación esperada o la estructura interna de texto es inválida. • Acciones técnicas del Sistema: 1. El sistema interrumpe inmediatamente la lectura del flujo de datos. 2. El backend captura el error, genera una excepción controlada y registra el evento. 3. Se purga y elimina automáticamente cualquier archivo temporal parcial creado en el servidor. 4. La interfaz bloquea el procesamiento y devuelve un mensaje de error modal al usuario: "Error: El formato del fichero no es correcto. Asegúrese de subir un documento de texto plano válido." • Estado final: La transacción se cancela y el sistema regresa al estado anterior sin alterar la base de datos.
Flujo alternativo 2 - Error de autenticación	• Condición de activación: Las credenciales del usuario no coinciden en la base de datos. • Acciones técnicas del Sistema: 1. El sistema interrumpe inmediatamente la llamada. 2. El backend captura el error, genera una excepción controlada y registra el evento. 3. La interfaz bloquea el procesamiento y devuelve un mensaje de error modal al usuario: "Error: Las credenciales no son válidas." • Estado final: La transacción se cancela y el sistema regresa al estado anterior sin alterar la base de datos.



Caso de uso: Consultar estadísticas de uso	
ID: CU-3	
Descripción	El gestor desea visualizar las estadísticas diarias del uso del parking.
Actores	Gestor
Precondiciones	- El sistema debe haber registrado datos históricos de ocupación. - El gestor debe estar autenticado en el panel de administración.
Flujo normal	- El gestor accede al panel de estadísticas. - El sistema consulta los datos históricos en la base de datos. - El backend procesa los datos y genera estadísticas (promedios, horas punta, ocupación media). - El frontend muestra gráficos o tablas con la información procesada.
Postcondiciones	El gestor visualiza correctamente las estadísticas del día o periodo seleccionado.
Flujo alternativo	- No existen registros suficientes → se muestra un mensaje indicando falta de datos. - Error en la conexión con la base de datos → se notifica al usuario.

Diagrama de clases

Se desarrollarán las siguientes clases:

- **Parking:** representa a cada uno de los parkings que puede gestionar la aplicación.
- **Plaza:** representa una plaza del parking, identificada por fila, columna y planta.
- **Registro:** guarda información de uso de una plaza.

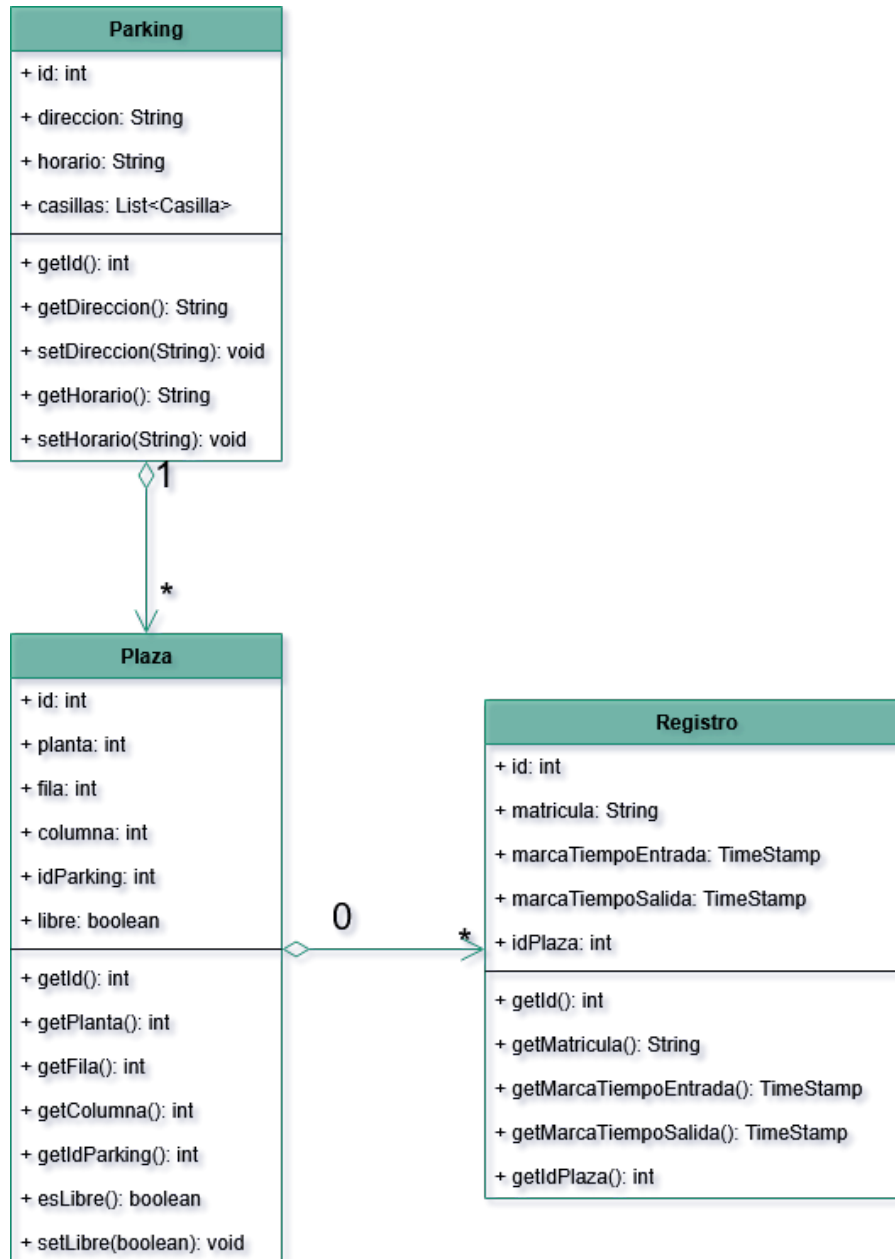
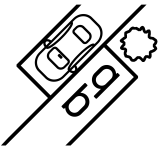
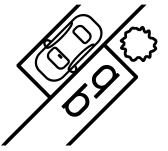


Diagrama de secuencia

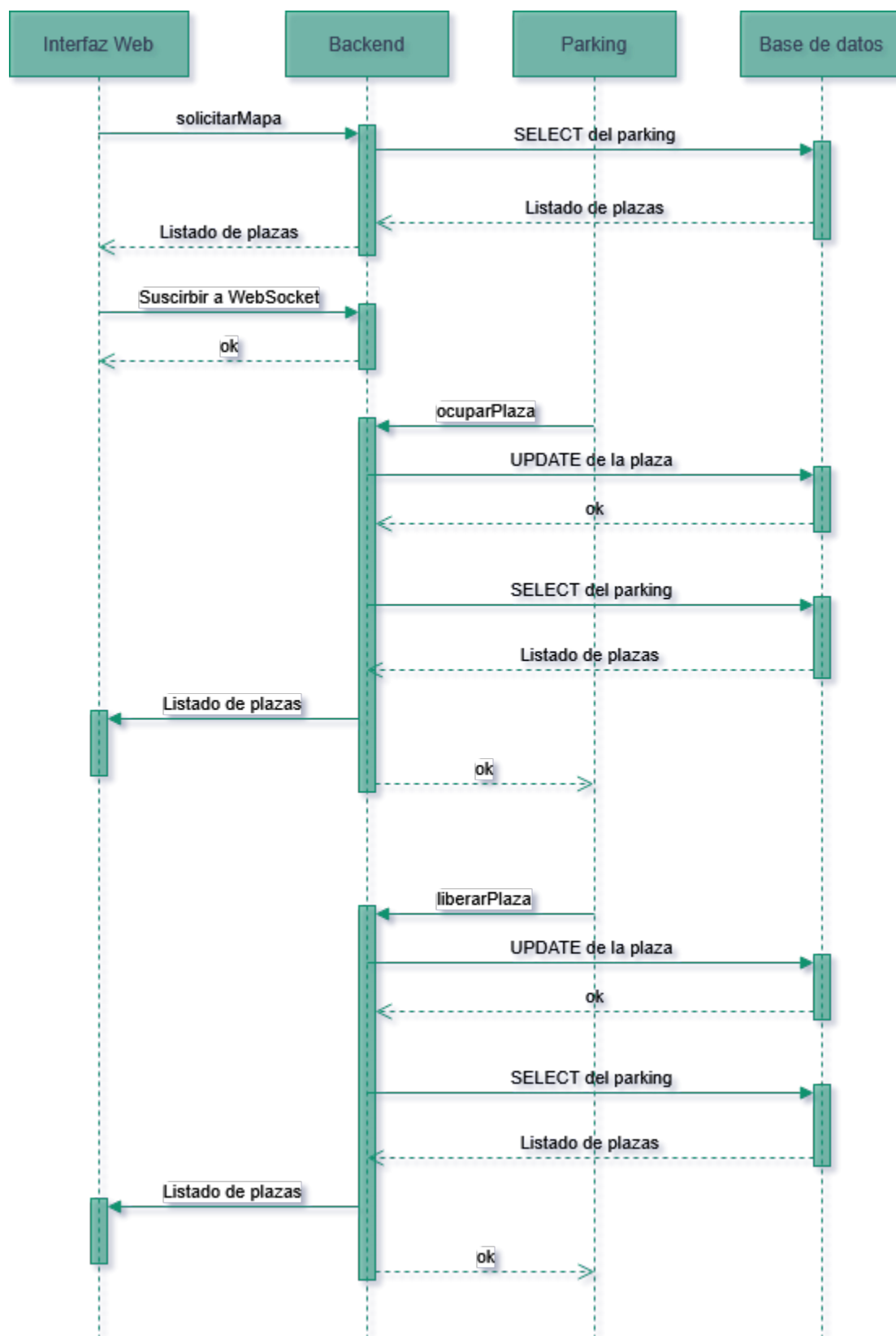
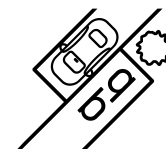
El siguiente diagrama de secuencia, muestra la comunicación entre los diferentes módulos de la aplicación, a la hora de mostrar el mapa de ocupación.

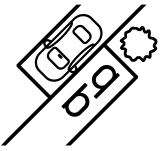
Al inicio, el **front-end** llama a la **API** para obtener el mapa de un parking, seguidamente se suscribe al **WebSocket**, para recibir actualizaciones cuando se **ocupan** o **liberan** plazas.



Cuando el sistema de sensores del parking detecta que un vehículo ocupa una plaza, llama a la API, indicando: id de la **plaza**, **matrícula** del vehículo y el **tipo** de este. A continuación el **back-end** se encargará de enviar el nuevo mapa a todos los usuarios **suscritos** al WebSocket del parking.

Sucede de forma similar al **liberar** plazas, pero en este caso, es suficiente con enviar el **identificador** de esta.





Diseño de interfaces

Visualizar mapa

- Muestra el plano del parking con sus casillas y plazas (colores según disponibilidad).
- Acciones disponibles:
 - Seleccionar una plaza → muestra información de la plaza
 - Botón **Menú** → abre un menú desplegable

Menú desplegable

- Muestra un menú desplegable
- Acciones disponibles:
 - Configuración → abre la configuración de la aplicación.
 - Menú de administración → abre la Pantalla de acceso al menú de administración

Pantalla de acceso al menú de administración

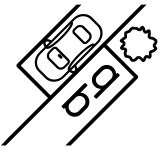
- Campos: usuario, contraseña.
- Acciones disponibles:
 - **Iniciar sesión** → lleva al menú de administración.

Menú de administración

- Muestra las diferentes opciones del menú de administración:
 - **Alta** de parking.
 - **Visualización** de estadísticas.

Alta de parking

- Permite dar de alta un parking a través de un fichero de texto.



Visualización de estadísticas

- Muestra las estadísticas de uso del parking a través de gráficos y tablas.

Material necesario para llevar a cabo el proyecto

Personal

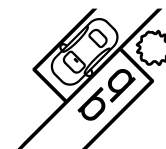
- 1 desarrollador backend de Java.
- 1 desarrollador frontend de React o Angular.
- 1 técnico en sistemas informáticos.
- 1 jefe de proyecto.
- 1 analista.

Equipo

- 4 ordenadores.
- 1 servidor para el frontend.
- 1 servidor para el backend.
- 1 servidor para base de datos.

Manuales

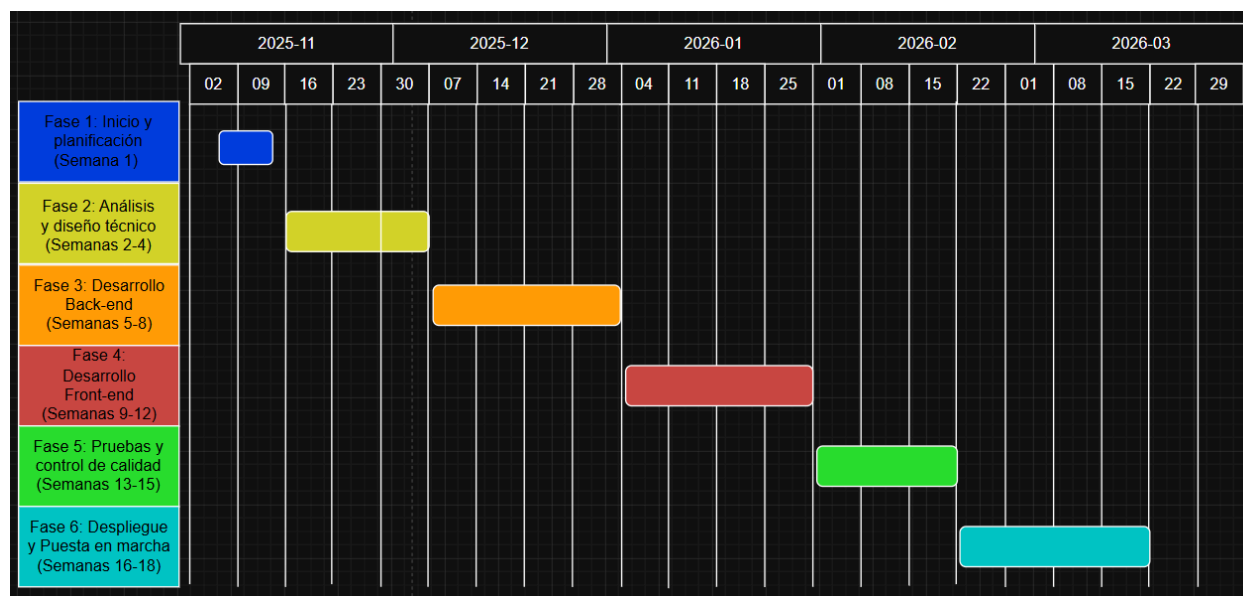
- Manual de gestor de la aplicación.
- Manual de usuario de la aplicación.
- Documentación técnica.



Parte III. Planificación y memoria económica

Gantt

El siguiente diagrama de gantt representa la planificación del proyecto entre noviembre de 2025 y marzo de 2026.



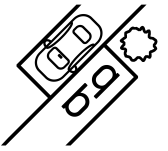
Recursos

1. Inicio del proyecto:

- Jefe de proyecto
- Analista

2. Análisis y diseño:

- Jefe de proyecto
- Analista
- Desarrollador de BBDD



- Desarrollador backend

3. Desarrollo backend:

- Jefe de proyecto
- Analista
- Desarrollador de BBDD
- Desarrollador backend

4. Desarrollo frontend:

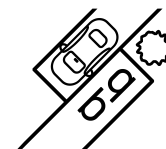
- Jefe de proyecto
- Desarrollador frontend

5. Integración y pruebas finales:

- Jefe de proyecto
- Analista
- Desarrollador de BBDD
- Desarrollador backend
- Desarrollador frontend
- Técnico de sistemas

6. Despliegue y documentación:

- Jefe de proyecto
- Analista
- Desarrollador de BBDD
- Desarrollador backend
- Desarrollador frontend



- Técnico de sistemas

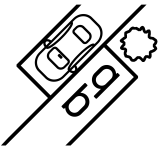
Costes

En la siguiente tabla se detalla el desglose de gastos del proyecto, dividido por las fases descritas en el diagrama de gantt. Los gastos son únicamente salarios, ya que todo el desarrollo de la aplicación se realiza sobre software libre/gratuito:

- Fase 1: Libre Office.
- Fase 2: Libre Office y [Draw.io](https://draw.io).
- Fase 3: MySQL, IDE Eclipse.
- Fase 4: VS Code.
- Fase 5: Navegador, Libre Office.
- Fase 6: Sistema remoto.

fase	semanas	personas	horas	coste
Inicio del proyecto	1	2	80	1840
Análisis y diseño	3	4	480	11040
Desarrollo backend	4	4	640	14720
Desarrollo frontend	4	2	320	7360
Integración y pruebas finales	3	6	720	16560
Despliegue y documentación	3	6	720	16560
Alquiler servidores	18			225
precio hora			COSTE TOTAL	68080
23				
alquiler servidores / mes				
50			TOTAL HORAS	2960

Aunque la fase de desarrollo e implementación se ha apoyado fuertemente en herramientas de software libre para minimizar los costes de licencias, la viabilidad y puesta en producción del sistema requiere una infraestructura de servidores, almacenamiento seguro y red. El coste total estimado para el desarrollo y despliegue, sumado a la carga salarial del desarrollo, asciende a **68080 €**.



Áreas de gestión del proyecto

Gestión de riesgos

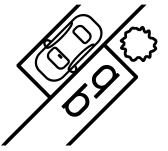
Se ha detectado una serie de riesgos que podrían afectar al desarrollo del proyecto, en la siguiente tabla se detallan, junto con el coeficiente de riesgo y un plan de contingencia para cada uno de ellos.

Riesgo	Probabilidad	Impacto	Coeficiente de riesgo (Pxl)	Plan de contingencia
Retraso en el desarrollo backend (Problemas con BBDD)	0,4	0,7	0,28	Priorizar módulos requeridos por front end
Retraso del desarrollo frontend (problemas de integración con backend)	0,3	0,5	0,15	Planear mas reuniones entre los desarrollaodres backend y frontend
Fallo del servidor durante las pruebas o despliegue	0,5	0,9	0,45	Utilizar contenedores Docker y crear copias de seguridad
Abandono temporal de un miembro del equipo (Baja o imprevisto)	0,7	0,8	0,56	Tener la documentación actualizada y contar con suplentes
Curva de aprendizaje en Vue.js / Spring	0,3	0,4	0,12	Dedicar suficiente tiempo en tutoriales y documentación de los framework
Pérdida o corrupción de datos de la Base de datos	0,4	0,9	0,36	Realizar copias de seguridad y exportaciones semanales a repositorios privados

Gestión del equipo del proyecto (personal)

Jefe de proyecto

- Coordinar reuniones y comunicación entre áreas.
- Aprobar entregas y documentación.
- Evaluar riesgos y aplicar medidas de contingencia.



- Definir objetivos y diagrama de Gantt.
- Comunicación con cliente.

Analista

- Comunicación con cliente.
- Realizar el análisis de requisitos.
- Supervisar que se cumplen los requisitos.

Desarrollador de BBDD

- Crear y mantener la base de datos.

Desarrollador backend

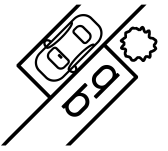
- Implementar la lógica de negocio en el backend.
- Diseñar e implementar la API REST.
- Dar seguridad al backend.

Desarrollador frontend

- Desarrollar los módulos de mapa en tiempo real, panel de administración y estadísticas.
- Integrar con la API REST del backend.
- Asegurar compatibilidad móvil y accesibilidad.

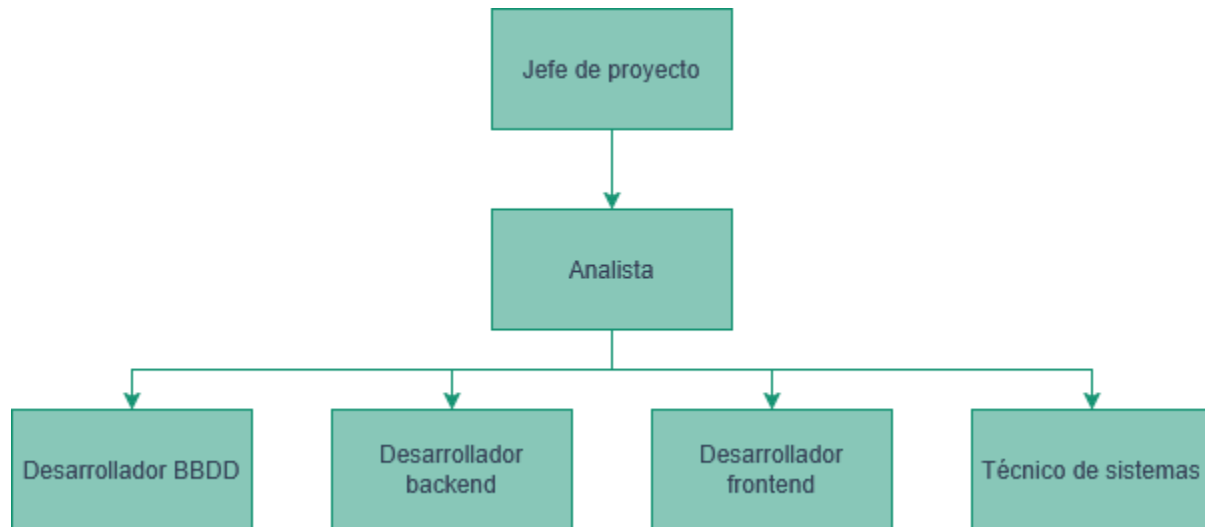
Técnico de sistemas

- Configurar entornos Docker y servidores.
- Administrar copias de seguridad.
- Gestionar despliegue y mantenimiento.
- Monitorear el rendimiento del sistema.



Organigrama

El organigrama quedaría de la siguiente forma:



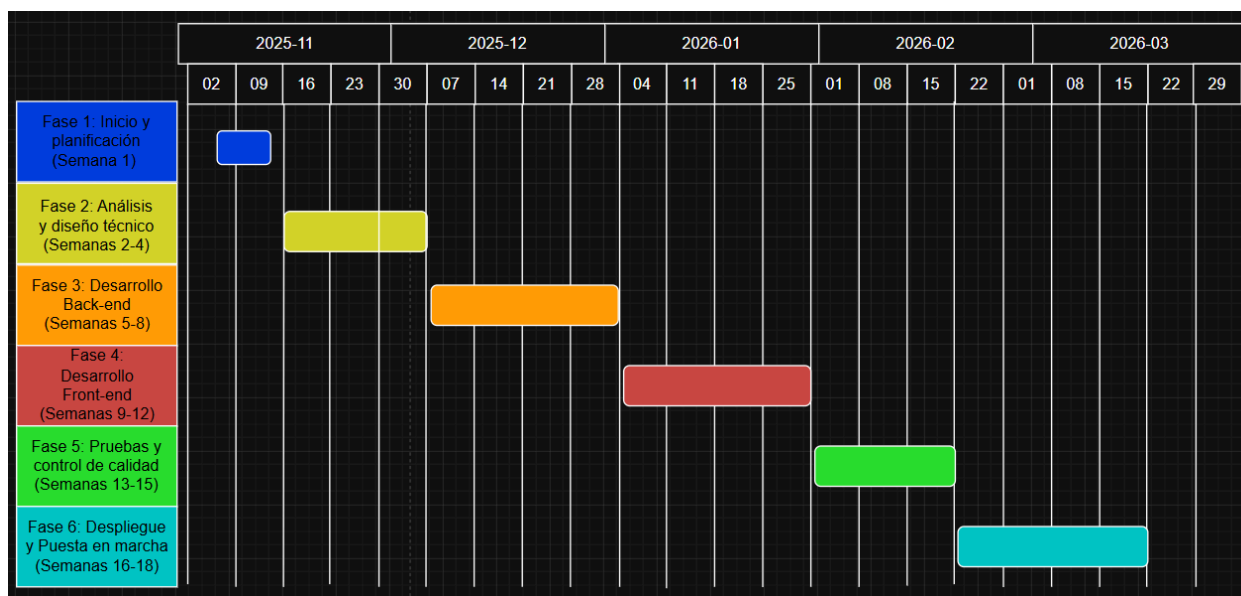
Parte III. Implementación y pruebas

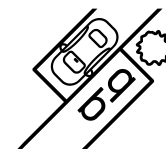
Grado de consecución producto

Comparativa Gantt inicio y final

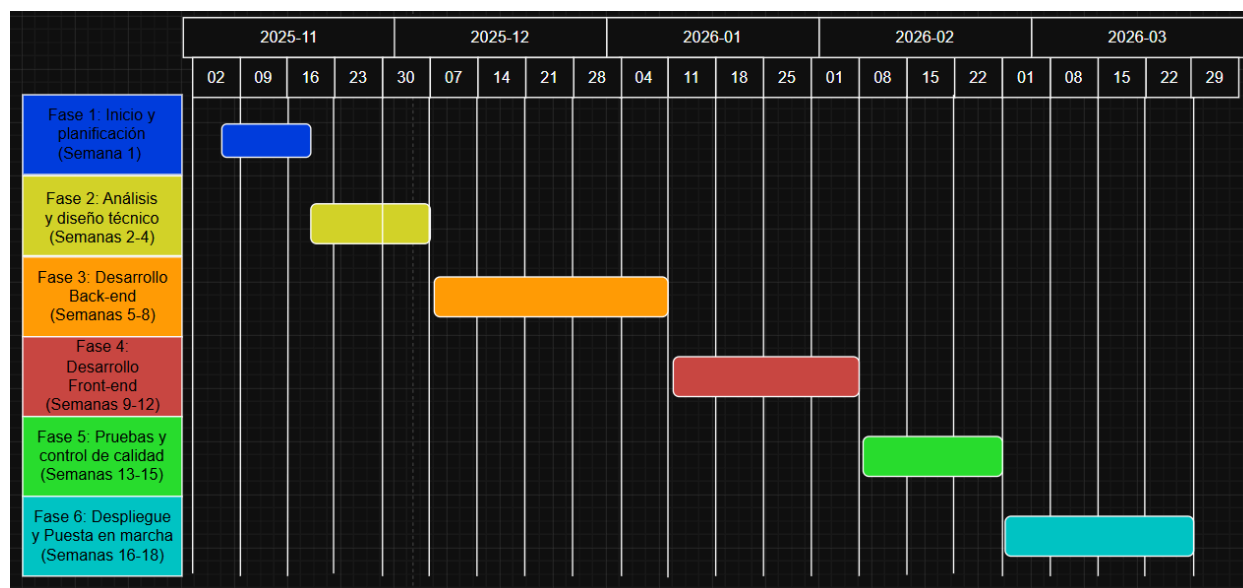
A continuación se muestra la comparativa entre el diagrama de gantt planteado al inicio del proyecto, y el diagrama real tras la realización de este.

Inicial:





Final:



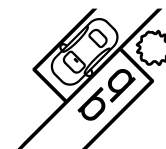
Sufrimos un retraso de una semana en el desarrollo backend, así que el proyecto se terminó en una semana más de los previsto.

Grado de cumplimiento de requisitos funcionales

- Funcionamiento general: fluido, sin tiempos de espera demasiado largos. Interfaz intuitiva y fácil de aprender.
- Visualización de parkings: Al entrar en la aplicación, entraremos en la página de inicio, en esta pantalla podremos ver todos los parking en una lista, y ver las plazas y el estado en el que está, del parking seleccionado.
- Alta de parking: Los administradores podrán añadir nuevos parking con un fichero, y adicionalmente, añadir más detalles en la web.
- Acceso a la aplicación: se accede mediante un navegador, para los gestores hay un sistema de validación de usuarios, para lo que necesitan introducir nombre del gestor y contraseña.

Existencia de un prototipo funcional

Existe un prototipo funcional en el entorno de preproducción, que permite:

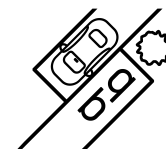


- Visualizar el mapa de ocupación de un parking virtual en tiempo real.
- Acceso al panel de administración.
- Subida de ficheros de configuración de nuevos parkings.
- Visualización de un dashboard de estadísticas.
- Validar credenciales de los administradores.
- Panel de configuración.

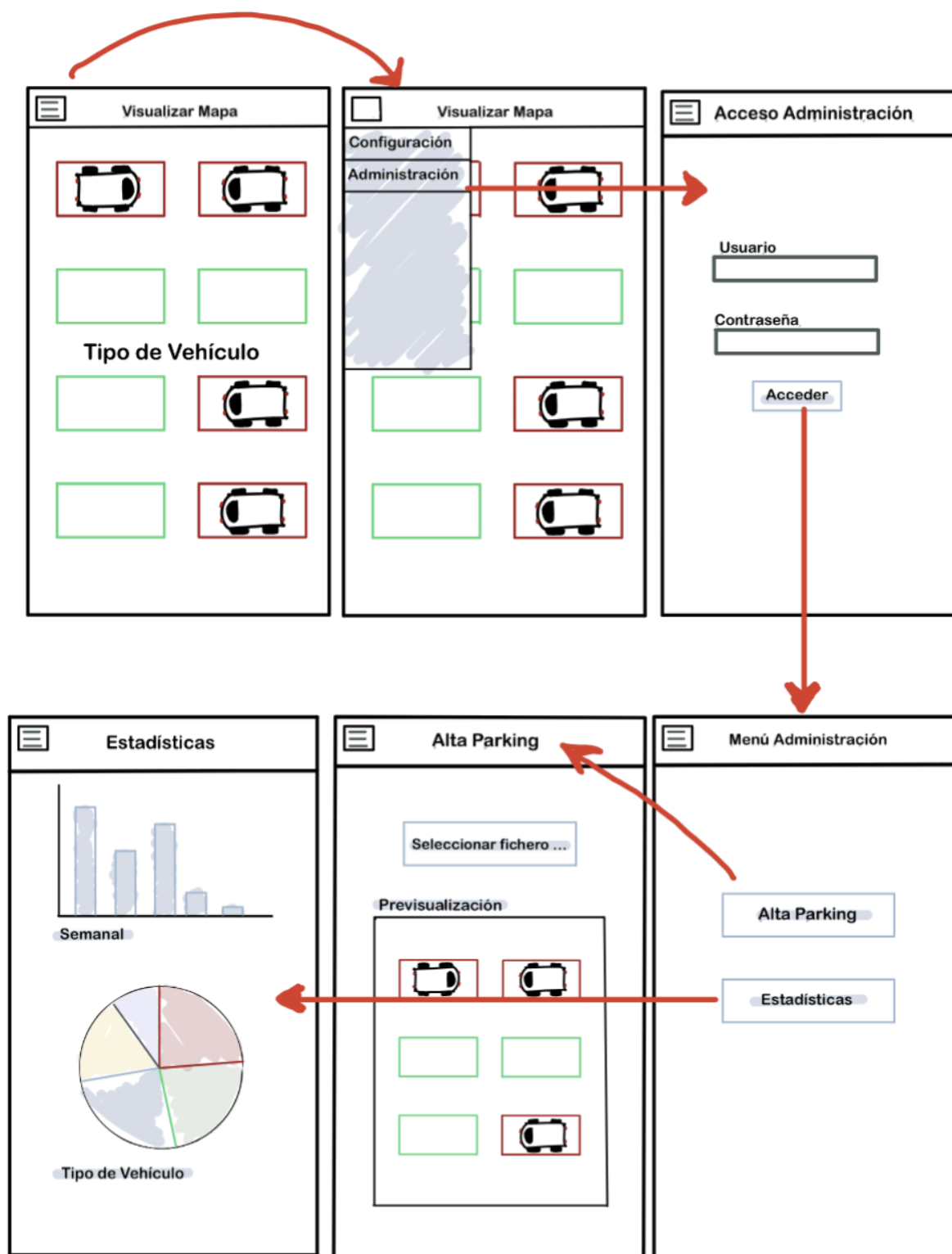
Pruebas realizadas

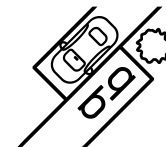
- Test unitarios en backend y frontend.
- Pruebas de carga en el servidor.
- Pruebas de almacenamiento en la base de datos.
- Pruebas de inicio de sesión para los administradores.
- Pruebas de extracción de datos desde el backend para obtener los parking.
- Prueba de carga de datos en tiempo real.

ID Prueba	Tipo de Prueba	Escenario / Condición	Resultado Esperado	Resultado Final
TEST-01	Unitaria	Inserción de datos límite en formularios	Bloqueo de caracteres no válidos	Éxito
TEST-02	De Carga	100 usuarios concurrentes enviando peticiones	Tiempo de respuesta estable sin caídas	Éxito
TEST-03	De Almacenamiento	Escritura de ficheros pesados en BBDD	Guardado íntegro sin corrupción de datos	Éxito



Mapa de navegación: Capturas de la aplicación y conexión entre ellas





Existen dos flujos principales según el tipo de usuario:

- **Cliente** del parking:
 1. El cliente abre la aplicación y visualiza el mapa de ocupación.
 2. El cliente pulsa el icono de menú y accede a la configuración.
 3. Personaliza los ajustes.
 4. Vuelve al mapa de ocupación.
- **Gestor** del parking:
 1. El gestor abre la aplicación y visualiza el mapa de ocupación.
 2. Pulsa el icono de menú y accede al login de administración.
 3. Introduce sus credenciales y pulsa en acceder.
 4. Visualiza el menú de administración:
 - a. Puede acceder al alta de nuevos parkings.
 - b. Puede acceder a las estadísticas de uso.

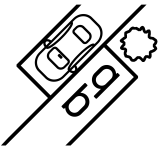
Innovación

Innovación

- Se han utilizado contenedores Docker para conseguir un despliegue más sencillo.
- Se han implementado sensores IOT conectados vía MQTT para obtener datos en tiempo real del parking.

Funcionalidades potenciales

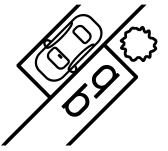
- Implementar un módulo de guiado con indicaciones a la plaza dentro del parking.
- Sistema de reserva de plazas.
- Servicio de análisis de datos para mejora y optimización del parking.
- Implementar pasarelas de pago.
- Adicionalmente a las pasarelas de pago, podemos automatizar pagos mediante el escaneo de la matrícula del vehículo o un QR.



Resultados y conclusiones

En el proceso nos hemos dado cuenta de la importancia que tiene analizar y planificar el diseño de la base de datos y cómo funcionan las comunicaciones entre las diferentes capas de la aplicación, que fue lo que nos retrasó tiempo, y así cumplir con el resultado deseado en el menor tiempo posible.

Aprendimos mucho trabajando en equipo y usando nuevas tecnologías. Conseguimos que el mapa del parking funcione en tiempo real y aunque tuvimos algunos problemas con la conexión y las estadísticas, logramos resolverlos. Al final obtuvimos un prototipo funcional que muestra bien nuestro esfuerzo y todo lo que aprendimos durante el proyecto.



Bibliografía - Webgrafía

Competencia

1. <https://www.parkopedia.es/>
2. <https://www.easypark.com/es-es>

Arquitectura

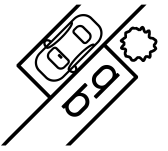
1. <https://www.ibm.com/mx-es/think/topics/three-tier-architecture>

Análisis técnico

1. <https://byspel.com/que-hace-un-tecnico-en-analisis-y-desarrollo-de-software/>

Desarrollo backend

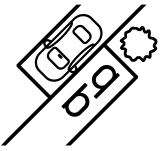
1. <https://spring.io/guides/gs/accessing-data-mysql>
2. <https://www.geeksforgeeks.org/advance-java/spring-boot-integrating-swagger-open-api/>
3. <https://www.baeldung.com/websockets-spring>
4. <https://www.baeldung.com/spring-websockets-send-message-to-user>
5. <https://stackoverflow.com/questions/36456458/simpmessagingtemplate-not-sending-messages-in-spring-boot>
6. <https://www.baeldung.com/spring-cors>
7. <https://stackoverflow.com/questions/14148880/how-do-i-reference-a-foreign-key-to-serial-datatype>
8. <https://www.datacamp.com/es/doc/mysql/mysql-create-view>
9. https://www.w3schools.com/mysql/mysql_join.asp



10. <https://thorben-janssen.com/spring-data-jpa-views/>

Desarrollo frontend

1. <https://www.w3schools.com/html/default.asp>
2. <https://www.w3schools.com/css/default.asp>
3. <https://www.w3schools.com/js/default.asp>
4. <https://www.w3schools.com/bootstrap5/index.php>
5. <https://getbootstrap.com/>
6. <https://www.w3schools.com/vue/index.php>
7. <https://vuejs.org/>



Anexos

Anexo 1 - Código SQL

```
DROP TABLE IF EXISTS gestor;
```

```
DROP TABLE IF EXISTS registro;
```

```
DROP TABLE IF EXISTS tipo_vehiculo;
```

```
DROP TABLE IF EXISTS plaza;
```

```
DROP TABLE IF EXISTS parking;
```

```
CREATE TABLE parking (
```

```
    id SERIAL PRIMARY KEY,
```

```
    direccion VARCHAR(255) NOT NULL,
```

```
    horario VARCHAR(100)
```

```
);
```

```
CREATE TABLE plaza (
```

```
    id SERIAL PRIMARY KEY,
```

```
    planta INTEGER NOT NULL,
```

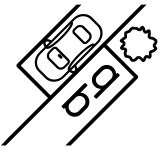
```
    fila INTEGER NOT NULL,
```

```
    columna INTEGER NOT NULL,
```

```
    id_parking INTEGER NOT NULL REFERENCES parking(id) ON DELETE CASCADE,
```

```
    libre BOOLEAN NOT NULL DEFAULT TRUE,
```

```
    UNIQUE (id_parking, planta, fila, columna)
```

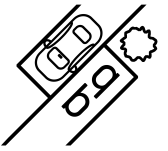


);

```
CREATE TABLE tipo_vehiculo (  
    id SERIAL PRIMARY KEY,  
    tipo VARCHAR(40) UNIQUE NOT NULL  
);
```

```
CREATE TABLE registro (  
    id SERIAL PRIMARY KEY,  
    matricula VARCHAR(20) NOT NULL,  
    id_tipo_vehiculo BIGINT UNSIGNED,  
    marca_tiempo_entrada TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    marca_tiempo_salida TIMESTAMP,  
    id_plaza INTEGER NOT NULL REFERENCES plaza(id) ON DELETE CASCADE,  
    FOREIGN KEY (id_tipo_vehiculo) REFERENCES tipo_vehiculo(id) ON DELETE CASCADE  
);
```

```
CREATE TABLE gestor (  
    id SERIAL PRIMARY KEY,  
    email VARCHAR(64) UNIQUE NOT NULL,  
    clave VARCHAR(16) NOT NULL,  
    nombre VARCHAR(20) NOT NULL,
```



```
apellidos VARCHAR(40) NOT NULL

);

CREATE OR REPLACE VIEW vista_registro AS

SELECT

r.id,

tipo AS tipo_vehiculo,

marca_tiempo_entrada,

marca_tiempo_salida,

id_parking

FROM registro r

JOIN tipo_vehiculo t ON r.id_tipo_vehiculo = t.id

JOIN plaza p ON r.id_plaza = p.id;
```